PRODUCTION-GRADE · END-TO-END · EVERY PRINCIPLE IMPLEMENTED

From Raw Transcripts to Board-Ready Intelligence

A walkthrough of every agentic AI architectural decision — theory, implementation, and production code — in one system built from scratch.

Claude Haiku 4.5 | NVIDIA NIM LLaMA-3.3-70B | LangGraph StateGraph | 7 Specialised Agents | 224 Unit Tests | Multi-Layer Security

Vinoth N
AI Systems Engineer · vinoth.n@outlook.com

Swipe to explore the architecture →

WHY THIS EXISTS

95% of customer calls are completely invisible

Contact centres manually analyse 2–5% of calls. The other 95% — coaching decisions, roadmap decisions, cost decisions — are made on sampled gut feel.

95%	70+	100%	<8m
CALLS NEVER REVIEWED	FIELDS EXTRACTED PER CALL	COVERAGE WITH THIS PIPELINE	TIME FOR 100 CALLS

This pipeline analyses **100% of calls, autonomously**. Every transcript becomes structured data. Every run produces board-level KPIs, cost-lever estimates, and AI-generated strategic recommendations — in minutes, end to end, with a full audit trail of every decision taken.

ANTHROPIC'S AGENTIC AI FRAMEWORK

Agents perceive, reason, act — and adapt

Agentic AI isn't a chatbot with tools. It's a system that autonomously plans, executes multi-step tasks, recovers from failures, and improves over time. Here's how each principle is implemented:

👁️ PERCEIVE

Theory: Ingest and validate environmental inputs
Implemented: DataIngestionAgent streams 3.7M-turn corpus, validates transcripts, scans for PII before any LLM sees the data

🧠 REASON

Theory: Plan actions through structured thinking
Implemented: Chain-of-Thought forces `_cot_reasoning` as the first JSON field before every extraction and recommendation

⚡ ACT

Theory: Execute tool calls and external API calls
Implemented: ReAct loop triggers targeted gap-fill retries; InsightsAgent calls NVIDIA NIM with 3-pass deliberation; ApprovalGate acts on operator input

📈 ADAPT

Theory: Learn from history and adjust behaviour
Implemented: Gemini-embedded vector store retrieves top-K similar historical runs; InsightsAgent receives trend context, not just current averages

LANGGRAPH STATEGRAPH · 7 SPECIALISED AGENTS

Every agent stateless, composable, replaceable

State flows immutably through the graph. Each agent receives the full `PipelineState` and returns an updated copy — decoupled, testable, and swappable without touching any other agent.

NORMAL PATH: ALL 7 NODES

QUALITY GATE FAILURE → ROUTES DIRECTLY TO EXPORT

224 UNIT TESTS · <7S CI

AGENTIC PRINCIPLE #1

Formal tool registry with per-agent access control

Every agent operation is registered as a named tool with a JSON schema. `AgentScopeGuard` enforces that agents can only call their authorised tools — a compromised or prompt-injected agent is blocked before the call executes.

PIPELINE/TOOLS.PY — TOOL REGISTRY

```
TOOL_REGISTRY = ToolRegistry()
TOOL_REGISTRY.register(Tool(
name="score_extraction", description="Score field
coverage", schema={"type": "object", ...},
func=score_field_coverage, )) # 5 tools: fetch ·
score · compute # insights · export
```

PIPELINE/SECURITY.PY — SCOPE GUARD

```
SCOPE_GUARD.check( "ExtractionAgent",
"score_extraction" ) # ✓ authorised
SCOPE_GUARD.check( "ExtractionAgent",
"export_results" ) # ✗ SecurityViolation
```

WHY THIS MATTERS

A prompt-injected ExtractionAgent that tries to call `export_results` — writing arbitrary data to disk — is blocked here before the call fires. Cross-agent tool hijacking is prevented at the framework layer, not by convention.

TOOL MANIFEST IS LLM-DISCOVERABLE

`TOOL_REGISTRY.manifest()` returns the full JSON-schema list — ready for any LLM that supports function calling to discover and invoke tools dynamically in future extensions.

AGENTIC PRINCIPLE #2

Observe → Reason → Act per transcript

Every extraction runs the ReAct control loop. If field coverage is below threshold, the agent reasons about which fields are missing and fires a targeted gap-fill call — not another full extraction.

👁️

OBSERVE

`score_field_coverage(result)`

Scores 7 critical fields: FCR, resolution, sentiment, duration, issues. Returns 0–100%.

🧠

REASON

Coverage < 70%? Identify exactly which critical fields are null. Build a targeted prompt — not a full re-extraction.

⚡

ACT

`gap_fill_transcript()`

Asks the LLM for *only* the missing fields. Merges result back — preserving all first-pass values.

```
# extraction_agent.py — _react_loop() coverage = score_field_coverage(result) # Observe if coverage <
REACT_QUALITY_THRESHOLD: # Reason improved = gap_fill_transcript( # Act client, system_prompt, transcript,
result ) react_stats['n_improved'] += 1
```

RECOVERS MISSING CRITICAL FIELDS

CIRCUIT-BREAKER ON QUOTA EXHAUSTION

UP TO REACT_MAX_ITERATIONS PER CALL

AGENTIC PRINCIPLE #3

Force reasoning before every extraction

The first field in every LLM response is `_cot_reasoning` — mandatory, enforced by the system prompt. The model must articulate the customer issue, resolution status, and sentiment trajectory *before* committing to any field value.

SYSTEM PROMPT INSTRUCTION

```
# prompts/system_prompt.txt "The FIRST field in your JSON MUST be `_cot_reasoning` — a 2-3 sentence step-by-step analysis covering: (1) the main customer issue (2) whether it was resolved (3) sentiment trajectory This reasoning MUST appear before all other fields in the JSON object."
```

EXAMPLE LLM OUTPUT STRUCTURE

```
{ "_cot_reasoning": "Customer called about billing discrepancy on mobile plan. Agent verified and corrected the charge. Sentiment improved.", "fcr": true, "resolution_status": "resolved", "sentiment_end": "positive" }
```

Why it works: Transformer attention is sensitive to token order. By forcing the model to reason first, subsequent fields are generated *conditioned on that reasoning* — not independently. This measurably reduces hallucination on enum and boolean fields at zero additional API cost.

APPLIED IN BOTH LLM AGENTS

ExtractionAgent — `_cot_reasoning` precedes all 70 extraction fields

InsightsAgent — Pass 1 prompt requires CoT KPI analysis before generating any recommendation

AGENTIC PRINCIPLE #4

The AI critiques itself before reaching executives

InsightsAgent runs a 3-pass deliberation loop. Pass 1 generates recommendations. Pass 2 self-grades each one (A/B/C) for specificity and data-grounding. Pass 3 rewrites anything that failed the critique.

PASS 1 — ANALYZE
Chain-of-Thought KPI analysis. Model reasons through each metric vs. industry benchmarks before generating 5 recommendations + 3 quick wins + 2 risk flags.

PASS 2 — CRITIQUE
A separate prompt grades every recommendation A/B/C on: data-grounded (traceable to a KPI?), specific (concrete action?), non-duplicated. Identifies missing angles.

PASS 3 — SYNTHESIZE
Rewrite every recommendation graded B or C. Final output must be directly traceable to a specific KPI. Generic platitudes are replaced with data-grounded actions.

```
# insights_agent.py — deliberation loop initial = _call_llm(ANALYZE_PROMPT, kpis) critique =
_call_llm(CRITIQUE_PROMPT, initial) final = _call_llm(SYNTHESIZE_PROMPT, initial, critique) # source:
"nvidia_nim_deliberation" | "claude_deliberation"
```

AGENTIC PRINCIPLE #5

Every autonomous decision records its WHY

`DecisionLogger` is wired into all 7 agents. Every routing choice, exclusion, provider selection, gap-fill trigger, and gate outcome is serialised to `decisions_{ts}.json` — retracing and challenging agent behaviour is always possible.

PIPELINE/DECISION_LOG.PY — USAGE

```
dl = DecisionLogger("QualityAgent", state) dl.log(
decision_type="qa_exclusion", decision="Excluded
call C-9821", reason="QA score 42 < threshold 60",
evidence={"score": 42, "threshold": 60},
confidence="high", alternatives=["Include with
warning"], ) return {**state, "decision_log":
dl.finalize()}
```

NAMED DECISION TYPES

TRANSCRIPT_SKIP · REACT_TRIGGER · QA_EXCLUSION · QUALITY_GATE_OUTCOME · PROVIDER_SELECTED · DELIBERATION_OUTCOME · ROUTING_DECISION · APPROVAL_DECISION

Enforced programmatically: Evidence dicts are validated at log time — forbidden keys (`transcript_text`, PII fields) raise `ValueError` before the record is written. The no-PII-in-decisions contract is a guard, not a convention.

AGENTIC PRINCIPLE #6

Agents learn from every run

Two memory layers work in parallel. Flat JSON stores run history and quota events. A semantic vector store embeds each run's KPI summary — InsightsAgent retrieves the most similar historical runs at inference time, not just averages.

PIPELINE/VECTOR_MEMORY.PY — SEMANTIC STORE

```
# After each run, ExportAgent stores KPIs
VECTOR_STORE.add_run( run_id="run_042",
kpi_text="FCR 72% AHT 4.2min avoidable 31%",
metadata={"fcr": 72, "aht": 4.2} ) # At inference,
InsightsAgent queries similar = VECTOR_STORE.query(
"high escalation low FCR run", top_k=3 ) # → top-3
most similar historical runs # injected into the
LLM prompt
```

EMBEDDING + RETRIEVAL

Primary: Gemini `text-embedding-004` (768-dim)
Fallback: TF-IDF bag-of-words (offline / CI)
Search: Cosine similarity — O(N) numpy, swappable to any vector DB
Backend-tagged: Each record carries its embedding backend — cross-backend queries are blocked to prevent garbage similarity scores

FLAT JSON MEMORY

Run history · cumulative KPIs · quota events · model performance stats — persisted to `outputs/agent_memory.json` and loaded at every pipeline start

DEFENCE IN DEPTH — PIPELINE/SECURITY.PY

Five guards at every agent boundary

Production autonomous systems face real attacks. Every input is sanitised before reaching an LLM. Every output is validated before reaching the next agent. Every API call is rate-capped. Every state serialisation scrubs secrets.

InputSanitizer
Size cap (token bomb defence), encoding cleanup, 9 prompt injection patterns neutralised, secret redaction in source transcripts

OutputSanitizer
Blocks code execution payloads (`eval` , `subprocess`), response bomb defence (32KB limit), field truncation at 2KB

AgentScopeGuard
Per-agent authorised tool sets enforced at call time. Cross-agent tool hijacking raises `SecurityViolation` before execution

SecretGuard
API key patterns (Anthropic, Google, NVIDIA, OpenAI, JWT, Bearer) blocked from LLM outputs and state serialisation. 7 pattern classes.

```
# RateLimiter — sliding window, per provider CLAUDE_RATE_LIMITER = RateLimiter(max_calls=50, window_s=60)
GEMINI_RATE_LIMITER = RateLimiter(max_calls=15, window_s=60) # Blocks runaway ReAct loops from burning quota
```

AGENTIC PRINCIPLE #7

Operator sign-off before autonomous export

The approval gate sits between InsightsAgent and ExportAgent. When enabled, it presents the KPI summary and waits for explicit operator confirmation before any outputs are written to disk. Auto-approves in CI. Always logs its decision.

PIPELINE/GRAPH.PY — APPROVAL_GATE_NODE

```
if not REQUIRE_HUMAN_APPROVAL: # Transparent pass-through in CI
dl.log(decision_type="approval_decision", decision="Auto-approved", ...) return {**state, "approval_granted": True} # Production: wait up to APPROVAL_TIMEOUT_S ready, _, _ = select.select( [sys.stdin], [], [], APPROVAL_TIMEOUT_S ) if not ready: granted = True # auto-approve on timeout
```

Why it's architecturally significant: Fully autonomous systems need a configurable override point. This gate is the pattern that lets the same pipeline run unattended in batch mode (CI) and supervised in regulated or high-stakes production environments — controlled by a single config flag.

CONFIG

`REQUIRE_HUMAN_APPROVAL = False` → silent CI mode
`REQUIRE_HUMAN_APPROVAL = True` → interactive prompt
`APPROVAL_TIMEOUT_S = 60` → auto-approve timeout

Every gate decision is recorded in DecisionLogger regardless of mode.

PIPELINE/GOVERNANCE.PY — FOUR GUARDRAILS

Cost · Quality · PII · Audit — without manual intervention

Every pipeline run passes through four governance checks. All are pure Python — fast, testable without API calls, and wired to `pipeline/config.py` so thresholds update in one place.

BUDGETGUARD

Hard cost cap per batch

Raises `BudgetExceededError` if inference cost exceeds the per-batch limit. Orchestrator splits `BUDGET_USD / n_batches` so 5 subprocesses cannot collectively spend 5× the global cap.

QUALITYGATE

Two-tier pass rate enforcement

Warning at 70% pass rate (log prominently, continue). Hard stop at 40% — catastrophic extraction failure routes pipeline directly to ExportAgent with a full audit trail.

PIISCANNER

Redact before LLM sees data

Six PII pattern classes: US phone, SSN, email, credit card, date of birth, account number. Scans every transcript before ExtractionAgent. Redacts with `[REDACTED]` token, logs PII types to AuditLog.

AUDITLOG

Structured append-only event trace

Records every agent start/end, tool call, governance check, and error with timestamps. Exported to `audit_log_{ts}.json` at pipeline end. Full decision trace for compliance and debugging.

PIPELINE/CONFIG.PY — SINGLE SOURCE OF TRUTH

One config change. Zero regressions.

Change `EXTRACTION_MODEL` in `config.py` — the right API client, rate limiter, cost accounting, key validation, and provider label all update automatically. No agent code changes required.

```
# pipeline/config.py — change ONE line EXTRACTION_MODEL = "claude-haiku-4-5-20251001" # → Anthropic client, 50RPM EXTRACTION_MODEL = "gemini-2.0-flash-lite" # → Google client, 15RPM EXTRACTION_MODEL = "claude-opus-4-8" # → Anthropic client, paid tier # Downstream: analyzer.py, token_tracker.py, security.py, # run_pipeline.py validation — ALL auto-update
```

EXTRACTION

Primary: Claude Haiku 4.5

Fallback: Gemini 2.0 Flash Lite

$0.0076 / call (Claude)

INSIGHTS

Primary: NVIDIA NIM LLaMA-3.3-70B

Fallback: Claude Haiku → Rule-based

OpenAI-compatible API

EMBEDDINGS

Primary: Gemini text-embedding-004

Fallback: TF-IDF (offline/CI)

768-dim vectors

FULL VISIBILITY ACROSS ALL 7 AGENTS

Every span captured. Every decision logged.

Three complementary observability layers provide complete visibility from individual LLM call to full pipeline run — without requiring any external service for the core audit trail.

LANGSMITH TRACING

One env var activates full span capture across all 7 LangGraph nodes and both LLM agents. View latency, token usage, and inputs/outputs per node in the LangSmith UI.

`LANGCHAIN_TRACING_V2=true`

DECISION LOG

15–60 structured WHY records per run — each with decision type, reason, evidence, confidence, alternatives. Serialised to `decisions_{ts}.json`. Queryable without any external tool.

AUDIT LOG

Every agent start/end, tool call, governance check, PII event, and error — timestamped and structured. `audit_log_{ts}.json` exported at run end.

```
outputs/ ├── decisions_{ts}.json ← 15-60 agent reasoning records ├── audit_log_{ts}.json ← full event trace (all agents) ├── run_manifest_{ts}.json ← batch metadata + checksums └── summary.json ← Streamlit dashboard source
```

100-CALL PRODUCTION RUN · CLAUDE HAIKU 4.5

Production metrics from a real pipeline run

224	70+	$0.008	90–99%
UNIT TESTS · <7S	FIELDS PER CALL	COST PER CALL	QA PASS RATE

KPI CATEGORIES EXTRACTED PER CALL

Resolution: FCR, resolution_status, escalated
Effort: handle_time_seconds, hold_time, issue_count
Intent: intent (24 categories), avoidable_call, ai_resolvable
Sentiment: start/end/trajectory
Risk: repeat_call_risk, churn_risk_signal
Commercial: upsell_attempted, cross_sell_opportunity
Reasoning: _cot_reasoning (step-by-step LLM analysis)

PIPELINE RUNTIME

~8 min for 100 calls (5×20 batches, 2s inter-call delay). Checkpoint/resume: kill at call 73, restart at 74 — zero duplicated API spend.

STREAMLIT DASHBOARD

8 sections: Hero · Cost Panels · Resolution Opportunity · AI Agent Roadmap · Actual Performance · Prioritised Action Plan · Performance Trends

EVERY TOOL CHOSEN FOR PRODUCTION-GRADE REASONS

The technology behind the system

No tutorial stack. Every choice reflects what a Series A AI company would actually ship — observable, testable, swappable, and governed.

🤖

Claude Haiku 4.5

Primary extraction LLM. CoT + ReAct. $0.008/call. 50 RPM rate-limited. Anthropic SDK.

⚡

NVIDIA NIM

LLaMA-3.3-70B for InsightsAgent deliberation. OpenAI-compatible API. Claude fallback → rule-based.

🕸️

LangGraph

StateGraph orchestration. 7 nodes. Conditional routing. LangSmith tracing. Immutable state handoff.

🔢

Gemini Embeddings

text-embedding-004, 768-dim. Vector memory for semantic KPI history retrieval. TF-IDF offline fallback.

📊

Streamlit

Executive dashboard. 8 sections. Reads live from summary.json. Falls back to demo data on cold start.

🧪

pytest · ruff · mypy

224 unit tests, <7s, zero API calls. Ruff linting + type checking gate every push to main.

OPEN TO THE RIGHT OPPORTUNITY

Building agentic AI is what I do.

This system demonstrates end-to-end ownership of a production-grade agentic AI platform — architecture, LLM integration, multi-agent orchestration, security, governance, observability, and shipping. If you're building something that needs this, let's talk.

EMPLOYEE
AI/ML Engineer · LLM Platform Engineer · Agentic Systems Engineer at an AI-first company

CO-FOUNDER
Technical co-founder for an agentic AI or enterprise SaaS venture — I bring the architecture

CONSULTANT
Agentic AI system design · LLM pipeline architecture · multi-agent frameworks

CONTRACT
Fixed-scope delivery: pipeline builds, LLM integrations, autonomous agent systems

📧 vinoth.n@outlook.com